Kausal Paths: Pipeline Operations Design Specification

Version: 0.1 (Draft) **Date**: December 2024 **Status**: Design phase

1. Executive Summary

This document specifies a declarative operation pipeline system for Kausal Paths, replacing the current ad-hoc `FormulaNode` implementation with a structured, inspectable, and extensible approach. The system enables users (including LLM-assisted authoring) to define custom node computations as sequences of typed operations.

Goals

- **Explainability**: Every computation step is inspectable with intermediate values visible for debugging and public transparency
- **Validation**: Catch errors at parse time through Pydantic schemas rather than runtime failures
- **Extensibility**: New operations can be added by defining a Pydantic model and an executor
- **LLM-friendly**: Structured format is easier for LLMs to generate correctly than free-form expressions
- **Hybrid authoring**: Support both declarative pipelines and formula expressions (compiled to pipelines)

Non-goals

- Real-time or sub-second computation (yearly resolution is sufficient)
- Replacing the DAG structure of nodes (pipelines operate *within* nodes)
- Supporting arbitrary Python code execution

---

2. Background and Context

Current State

Kausal Paths models are directed acyclic graphs (DAGs) where nodes represent computations on dimensional time series data. Currently, custom computations are handled through:

1. **Typed node classes** (e.g., `VehicleMileage`, `MultiplicativeNode`) - Python classes with hardcoded logic

2. **FormulaNode** - Parses Python-like expressions using `ast` and evaluates supported operations manually
3. **Ad-hoc operation registries** - Two separate registries with overlapping, organically-grown operations

The current FormulaNode implementation:

```python
def eval_tree(self, tree: ast.AST, varss: EvalVars) -> EvalOutput:
    EVALUATORS: dict[type, Callable[[Any, EvalVars], EvalOutput]] = {
        ast.Expression: self.eval_expression,
        ast.Constant: self.eval_constant,
        ast.Name: self.eval_name,
        ast.BinOp: self.eval_binop,
        ast.Call: self.eval_call,
    }
    # ...
```

Problems with Current Approach

1. **No intermediate value inspection** - Debugging requires print statements or stepping through code
2. **Implicit dependencies** - Hard to determine what inputs a formula actually uses
3. **No validation** - Errors surface at runtime, often with unclear messages
4. **Two separate operation sets** - Node operations vs DataFrame extension operations, with no unified model
5. **Domain-specific hacks** - Operations like `skip_dim_test`, `ignore_content` suggest workarounds rather than clean abstractions

The Opportunity

A declarative pipeline system provides:

- Built-in explainability (every step has inputs, outputs, and description)
- Schema validation via Pydantic
- Easy serialization to/from YAML, JSON, and database storage
- Natural fit for visual graph editing UI
- Foundation for LLM-assisted model building

---

3. Core Concepts

3.1 Data Model: PathsDataFrame

All data flowing through pipelines is represented as `PathsDataFrame` (PDF), a Polars DataFrame subclass with additional metadata:

Column	Type	Description
`Year`	Int	The time dimension (yearly resolution)
`Forecast`	Bool	Whether this row is historical data or forecast
`Value`	Float	The metric value
dimension columns	Categorical	Zero or more dimensional breakdowns

Metadata carried by PDF:

- `unit`: Physical unit (pint-compatible string, e.g., `"kt CO2/a"`, `"EUR"`, `"DALY/a"`)
- `quantity`: Semantic quantity type (e.g., `"emissions"`, `"cost"`, `"health_effect"`)

Example DataFrame:

```
┌──────┬──────────┬────────────┬──────────┐
│ Year │ Forecast │ vehicle_type │ Value    │
│ i32  │ bool     │ cat        │ f64      │
╞══════╪══════════╪════════════╪══════════╡
│ 2020 │ false    │ cars       │ 12500.0 │
│ 2020 │ false    │ buses      │ 850.0   │
│ 2021 │ false    │ cars       │ 12100.0 │
│ 2021 │ false    │ buses      │ 870.0   │
│ 2025 │ true     │ cars       │ 10500.0 │
│ 2025 │ true     │ buses      │ 920.0   │
└──────┴──────────┴────────────┴──────────┘
Unit: Mvkm/a
Quantity: vehicle_mileage
```

3.2 Nodes and Ports

Nodes are the vertices in the model DAG. Each node has:

- **Input ports**: Named connection points receiving data from upstream nodes or datasets

- **Output ports**: Named connection points providing results to downstream nodes
- **Computation**: Either a typed node class OR an operation pipeline

```yaml
# Example node with pipeline computation
- id: net_emissions
  name_en: Net emissions after reductions
  input_ports:
    - id: gross
      quantity: emissions
      unit: kt CO2/a
    - id: reductions
      quantity: emissions
      unit: kt CO2/a
  output_ports:
    - id: output
      quantity: emissions
      unit: kt CO2/a
      dimensions: [sector]
  operations:
    - kind: subtract
      from: { port: gross }
      values: [{ port: reductions }]
    - kind: clip
      input: { ref: subtract_result }
      min: 0
      description: "Emissions cannot go negative"
```

3.3 Edges and Edge Transformations

Edges connect output ports to input ports. They can carry **structural transformations** that filter or reshape data without changing values:

- **Select categories**: Filter to specific dimension values
- **Exclude categories**: Filter out specific dimension values
- **Assign category**: Add a dimension with a fixed value
- **Flatten**: Sum over a dimension (remove it)

Design principle: Edge transformations handle *what data flows and in what shape*. Node operations handle *what happens to values*.

The test: "Does this change the numbers, or just which numbers are visible?" If just visibility/shape → edge. If values change → node operation.

3.4 Pipelines

A pipeline is an ordered sequence of operations that transforms input port data into output port data. Each operation:

- References inputs (ports, datasets, intermediate results, parameters, or scalar constants)
- Produces an output (named intermediate or implicit final result)
- Has an optional human-readable description
- May have conditional execution modifiers (only_if/skip_if)

The final operation's output becomes the node's output.

3.5 Parameters and Scenarios

Parameters enable user-configurable model behavior without modifying the model structure. They come in two scopes:

Global parameters: Model-wide settings that affect multiple nodes

- Example: `discount_rate`, `green_electricity_emissionless`, `target_year`

Node-local parameters: Settings specific to a single node

- Example: Action nodes have `enabled` parameter
- Example: A specific reduction action might have `reduction_percentage`

A **scenario** is a collection of parameter values. Users can:

- Use predefined scenarios (e.g., "Business as usual", "Ambitious", "Net Zero")
- Create custom scenarios by adjusting parameter values via UI (sliders, toggles)
- Compare outcomes across different scenarios

Parameters in pipelines: Operations can reference parameters as inputs, allowing calculations to respond to scenario settings. For example:

```yaml
```

```yaml
# Discount rate from scenario parameter
- kind: discount
  input: { port: future_costs }
  rate: { parameter: discount_rate }

# Conditional logic based on policy assumption
- kind: set_values
  input: { port: emission_factor }
  value: 0
  only_if: { parameter: green_electricity_emissionless }
```

Note: Parameter definitions (type, default value, constraints, UI hints) are managed separately from the pipeline specification. Pipelines only reference parameters by ID.

---

4. Type Definitions

4.1 Basic Types

```python
```

```python
from pydantic import BaseModel, Field
from typing import Literal, Annotated, Union
from enum import Enum
import re

# Identifier pattern: lowercase alphanumeric with underscores and hyphens
IDENTIFIER_RE = r'^[a-z][a-z0-9_-]*$'
Identifier = Annotated[str, Field(pattern=IDENTIFIER_RE)]

class I18nString(BaseModel):
    """Internationalized string with language variants."""
    en: str
    fi: str | None = None
    de: str | None = None
    # ... other languages as needed

class Unit(BaseModel):
    """
    Physical unit specification.

    The unit string must be pint-compatible. The optional specifier
    provides human-readable context (e.g., "of CO2", "of apple juice")
    for preventing nonsensical additions even when base units match.
    """
    unit: str
    specifier: I18nString | None = None

    def is_compatible_with(self, other: 'Unit') -> bool:
        """Check if units can be combined (same dimensions and specifier)."""
        # Implementation uses pint for dimensional analysis
        # and compares specifiers if present
        ...
```

4.2 Input References

Operations reference their inputs through a discriminated union:

```python
```

```python
class PortRef(BaseModel):
    """Reference to a node input port."""
    port: Identifier

class IntermediateRef(BaseModel):
    """Reference to an intermediate result from earlier in the pipeline."""
    ref: Identifier

class DatasetRef(BaseModel):
    """Reference to a dataset by ID."""
    dataset: Identifier

class ParameterRef(BaseModel):
    """
    Reference to a parameter value from the active scenario.

    Parameters can be global (model-wide) or node-local.
    The parameter definition (type, default, constraints) is managed
    separately; this just references by ID.
    """
    parameter: Identifier

class ScalarValue(BaseModel):
    """
    A constant scalar value with unit.

    Use `dimensionless: true` for pure ratios (e.g., 0.85 efficiency factor).
    Otherwise, unit is required.
    """
    value: float
    unit: str | None = None
    dimensionless: bool = False

    @model_validator(mode='after')
    def check_unit_or_dimensionless(self) -> Self:
        if not self.dimensionless and self.unit is None:
            raise ValueError("Either unit or dimensionless=true must be specified")
        return self

# The union type for any input reference
InputRef = Annotated[
    Union[PortRef, IntermediateRef, DatasetRef, ParameterRef, ScalarValue],
    Field(discriminator=None)  # Discriminate by which field is present
]
```

```
# Boolean parameter reference (for conditional execution)
BoolParameterRef = ParameterRef  # Type alias for clarity; must resolve to boolean
```

YAML examples:

```yaml
```

```yaml
# Reference to input port
- kind: multiply
  inputs:
    - port: vehicle_kilometers
    - port: emission_factor

# Reference to intermediate result
- kind: clip
  input:
    ref: gross_emissions
  min: 0

# Scalar with unit
- kind: multiply
  inputs:
    - port: energy_consumption
    - value: 0.21
      unit: kg CO2/kWh

# Dimensionless scalar (ratio)
- kind: multiply
  inputs:
    - port: baseline
    - value: 0.85
      dimensionless: true

# Reference to a parameter (value comes from active scenario)
- kind: discount
  input:
    port: future_costs
  rate:
    parameter: discount_rate

# Global parameter controlling calculation method
- kind: set_values
  input:
    port: grid_emission_factor
  value: 0
  only_if:
    parameter: green_electricity_emissionless
```

4.3 Base Operation

```python
```

```python
class BaseOperation(BaseModel):
    """
    Base class for all pipeline operations.

    All operations inherit from this and add their specific fields.
    The `kind` field acts as discriminator for the union type.
    """
    kind: str  # Overridden in subclasses with Literal type

    output: Identifier | None = None
    """
    Identifier for this operation's result.
    If None, this is the final operation and produces the node output.
    """

    description: I18nString | str | None = None
    """
    Human-readable explanation of what this operation does.
    Displayed in UI and useful for explainability.
    """

    only_if: ParameterRef | None = None
    """
    Conditional execution: only run this operation if the boolean parameter is true.
    If false, the operation is skipped and the primary input passes through unchanged.
    Only valid for single-input operations.
    """

    skip_if: ParameterRef | None = None
    """
    Conditional execution: skip this operation if the boolean parameter is true.
    If true, the operation is skipped and the primary input passes through unchanged.
    Only valid for single-input operations.
    Mutually exclusive with only_if.
    """

    @model_validator(mode='after')
    def check_conditional_exclusivity(self) -> Self:
        if self.only_if is not None and self.skip_if is not None:
            raise ValueError("Cannot specify both only_if and skip_if on same operation")
        return self

    class Config:
        extra = "forbid"  # Catch typos in YAML
```

Conditional Execution Semantics

The `only_if` and `skip_if` modifiers enable conditional operation execution without introducing nested pipelines:

Modifier	Parameter Value	Behavior
`only_if`	true	Execute operation normally
`only_if`	false	Skip operation, pass through primary input
`skip_if`	true	Skip operation, pass through primary input
`skip_if`	false	Execute operation normally

Constraints:

- Only valid on single-input operations (must have unambiguous passthrough)
- The referenced parameter must resolve to a boolean value
- Cannot use both `only_if` and `skip_if` on the same operation

Example use cases:

```yaml
# Zero out emission factor based on policy assumption
- kind: set_values
  input: { port: grid_emission_factor }
  value: 0
  only_if: { parameter: green_electricity_emissionless }

# Skip cumulative calculation based on display preference
- kind: cumsum
  input: { ref: annual_emissions }
  skip_if: { parameter: show_annual_values }

# Conditionally apply a reduction factor
- kind: multiply
  inputs:
    - ref: baseline
    - value: 0.8
      dimensionless: true
  output: reduced
  # Note: Cannot use only_if/skip_if here - multiple inputs
```

5. Operation Catalog

5.1 Arithmetic Operations

Add

```python
class Add(BaseOperation):
    """
    Element-wise addition of two or more inputs.

    All inputs must have compatible units (same physical dimensions).
    Dimensions are broadcast automatically.
    """
    kind: Literal["add"] = "add"
    inputs: list[InputRef] = Field(min_length=2)
```

Unit behavior: All inputs must have identical units. Output has same unit.

Dimension behavior: Union of all input dimensions. Missing dimension values are broadcast.

Subtract

```python
class Subtract(BaseOperation):
    """
    Subtract values from a base value.

    Computes: from - sum(values)
    Equivalent to: from - values[0] - values[1] - ...
    """
    kind: Literal["subtract"] = "subtract"
    from_: InputRef = Field(alias="from")
    values: list[InputRef] = Field(min_length=1)
```

Note: Field named `from_` with alias `"from"` because `from` is a Python keyword.

Unit behavior: All inputs must have identical units. Output has same unit.

Multiply

```python
```

```python
class Multiply(BaseOperation):
    """
    Element-wise multiplication of two or more inputs.

    Units are multiplied according to dimensional analysis.
    """
    kind: Literal["multiply"] = "multiply"
    inputs: list[InputRef] = Field(min_length=2)
```

Unit behavior: Units multiply. E.g., `km * kg/km = kg`.

Divide

```python
class Divide(BaseOperation):
    """
    Element-wise division: numerator / denominator.

    Only supports two operands for clarity.
    For a / (b * c), use explicit multiplication first.
    """
    kind: Literal["divide"] = "divide"
    numerator: InputRef
    denominator: InputRef
```

Unit behavior: Units divide. E.g., `kg / km = kg/km`.

Error handling: Division by zero produces NaN with warning.

Power

```python
class Power(BaseOperation):
    """
    Raise base to an exponent.

    Exponent should typically be dimensionless.
    """
    kind: Literal["power"] = "power"
    base: InputRef
    exponent: InputRef | float
```

Unit behavior: Unit is raised to the power. E.g., `m^2` when squaring meters.

Negate

```python
class Negate(BaseOperation):
    """
    Multiply by -1. Convenience for sign flipping.
    """
    kind: Literal["negate"] = "negate"
    input: InputRef
```

5.2 Clamping and Bounds

Clip

```python
class Clip(BaseOperation):
    """
    Constrain values to [min, max] range.

    At least one of min or max must be specified.
    Values outside range are set to the boundary value.
    """
    kind: Literal["clip"] = "clip"
    input: InputRef
    min: float | None = None
    max: float | None = None

    @model_validator(mode='after')
    def check_bounds(self) -> Self:
        if self.min is None and self.max is None:
            raise ValueError("At least one of min or max must be specified")
        if self.min is not None and self.max is not None and self.min > self.max:
            raise ValueError("min must be <= max")
        return self
```

Common patterns:

- `clip(min=0)` - Make non-negative (replaces `make_nonnegative`)
- `clip(max=0)` - Make non-positive (replaces `make_nonpositive`)
- `clip(min=0, max=1)` - Constrain to valid share/fraction range

Coalesce

```python
class Coalesce(BaseOperation):
    """
    Replace null/NaN values with a fallback.
    """
    kind: Literal["coalesce"] = "coalesce"
    input: InputRef
    fallback: InputRef | float = 0.0
```

Replaces `empty_to_zero`, `drop_nans` patterns.

5.3 Cumulative and Sequential Operations

These operations work along the time (Year) axis.

CumulativeSum

```python
class CumulativeSum(BaseOperation):
    """
    Running sum over time axis.

    For each year, output is sum of all values from start up to that year.
    """
    kind: Literal["cumsum"] = "cumsum"
    input: InputRef
```

Unit behavior: Changes unit from rate to cumulative. E.g., `t CO2/a` → `t CO2`.

CumulativeProduct

```python
class CumulativeProduct(BaseOperation):
    """
    Running product over time axis.

    Useful for compound growth calculations.
    """
    kind: Literal["cumprod"] = "cumprod"
    input: InputRef
```

Difference

```python
class Difference(BaseOperation):
    """
    Year-over-year difference.

    Output for year Y is: value[Y] - value[Y - periods]
    First `periods` years will have NaN values.
    """
    kind: Literal["diff"] = "diff"
    input: InputRef
    periods: int = 1
```

Shift

```python
class Shift(BaseOperation):
    """
    Shift values forward or backward in time.

    Positive periods shift forward (value at year Y comes from Y - periods).
    Negative periods shift backward.
    """
    kind: Literal["shift"] = "shift"
    input: InputRef
    periods: int
```

RollingMean

```python
class RollingMean(BaseOperation):
    """
    Rolling average over a window of years.
    """
    kind: Literal["rolling_mean"] = "rolling_mean"
    input: InputRef
    window: int = Field(ge=1)
    min_periods: int | None = None  # Minimum observations required
```

5.4 Time Filtering

TimeFilter

```python
class TimeFilterMode(str, Enum):
    ALL = "all"
    FORECAST_ONLY = "forecast_only"
    HISTORICAL_ONLY = "historical_only"

class TimeFilter(BaseOperation):
    """
    Filter data to specific time ranges or forecast/historical status.
    """
    kind: Literal["time_filter"] = "time_filter"
    input: InputRef
    mode: TimeFilterMode = TimeFilterMode.ALL
    year_min: int | None = None
    year_max: int | None = None
```

Replaces `forecast_only`, `inventory_only`, `truncate_before_start`, `truncate_beyond_end`.

5.5 Interpolation and Extrapolation

Interpolate

```python
class InterpolationMethod(str, Enum):
    LINEAR = "linear"
    NEAREST = "nearest"
    FORWARD_FILL = "forward"
    BACKWARD_FILL = "backward"
    ZERO = "zero"

class Interpolate(BaseOperation):
    """
    Fill missing years within the data range.

    Does not extrapolate beyond existing data bounds.
    """
    kind: Literal["interpolate"] = "interpolate"
    input: InputRef
    method: InterpolationMethod = InterpolationMethod.LINEAR
```

Extrapolate

```python
```

```python
class ExtrapolationMethod(str, Enum):
    CONSTANT = "constant"  # Extend last known value
    LINEAR = "linear"      # Linear trend from last N points
    GROWTH_RATE = "growth_rate"  # Compound growth from historical rate

class Extrapolate(BaseOperation):
    """
    Extend series beyond known data range.

    Typically used to fill forecast years based on historical patterns.
    """
    kind: Literal["extrapolate"] = "extrapolate"
    input: InputRef
    method: ExtrapolationMethod = ExtrapolationMethod.CONSTANT
    target_year: int | None = None  # Extend to this year
    trend_years: int = 5  # Years of history to use for trend calculation
```

FillMissingYears

```python
class FillMissingYears(BaseOperation):
    """
    Ensure continuous yearly data by adding missing years.

    Added years get NaN values (combine with Interpolate to fill).
    """
    kind: Literal["fill_missing_years"] = "fill_missing_years"
    input: InputRef
    start_year: int | None = None
    end_year: int | None = None
```

Replaces `add_missing_years`.

5.6 Dimensional Operations

SelectCategories

```python
```

```python
class SelectCategories(BaseOperation):
    """
    Filter to specific categories within a dimension.

    Note: Consider using edge transformations for simple filtering
    at node boundaries. Use this operation for mid-pipeline filtering.
    """
    kind: Literal["select"] = "select"
    input: InputRef
    dimension: Identifier
    categories: list[str]
    exclude: bool = False  # If true, select everything EXCEPT these categories
```

AggregateOver

```python
class AggregationMethod(str, Enum):
    SUM = "sum"
    MEAN = "mean"
    MIN = "min"
    MAX = "max"
    FIRST = "first"
    LAST = "last"

class AggregateOver(BaseOperation):
    """
    Aggregate (collapse) one or more dimensions.

    Note: For simple summing at node boundaries, consider using
    edge transformation flatten instead.
    """
    kind: Literal["aggregate"] = "aggregate"
    input: InputRef
    dimensions: list[Identifier]
    method: AggregationMethod = AggregationMethod.SUM
```

SplitByShares

```python
```

```python
class SplitByShares(BaseOperation):
    """
    Distribute totals across a dimension using share weights.

    The shares input must have the target dimension.
    Shares should sum to 1.0 for each year (validated with warning if not).
    """
    kind: Literal["split_by_shares"] = "split_by_shares"
    totals: InputRef
    shares: InputRef
    target_dimension: Identifier
```

Broadcast

```python
class Broadcast(BaseOperation):
    """
    Expand data to include a new dimension by repeating values.

    The categories_from input defines the categories for the new dimension.
    """
    kind: Literal["broadcast"] = "broadcast"
    input: InputRef
    dimension: Identifier
    categories_from: InputRef  # Get categories from this input's dimension
```

5.7 Financial Operations

These support Phase 2 expansion into broader strategic planning.

Discount

```python
class Discount(BaseOperation):
    """
    Apply time-value discounting to calculate present value.

    Discounts future values back to base_year using the specified rate.
    """
    kind: Literal["discount"] = "discount"
    input: InputRef
    rate: InputRef | float  # Annual discount rate (e.g., 0.03 for 3%)
    base_year: int | None = None  # Reference year; defaults to first year in data
```

Unit behavior: Preserves unit but values represent present value.

Annualize

```python
class Annualize(BaseOperation):
    """
    Convert one-time capital costs to equivalent annual costs.

    Spreads a lump sum over the asset lifetime using the discount rate.
    """
    kind: Literal["annualize"] = "annualize"
    input: InputRef
    lifetime_years: InputRef | int
    discount_rate: InputRef | float = 0.03
```

CompoundGrowth

```python
class CompoundGrowth(BaseOperation):
    """
    Apply compound growth rate from a base year.

    Output[Y] = Input[base_year] * (1 + rate)^(Y - base_year)
    """
    kind: Literal["compound_growth"] = "compound_growth"
    input: InputRef
    rate: InputRef | float  # Annual growth rate
    base_year: int | None = None
```

5.8 Conditional Operations

Where

```python
```

```python
class ComparisonOperator(str, Enum):
    EQ = "eq"    # ==
    NE = "ne"    # !=
    LT = "lt"    # <
    LE = "le"    # <=
    GT = "gt"    # >
    GE = "ge"    # >=

class Where(BaseOperation):
    """
    Conditional value selection.

    For each element: if condition is true, use if_true value, else if_false.
    """
    kind: Literal["where"] = "where"
    condition_input: InputRef
    condition_op: ComparisonOperator
    condition_value: InputRef | float
    if_true: InputRef
    if_false: InputRef
```

Coalesce (already defined above)

Used for "first non-null" patterns.

5.9 Utility Operations

SetValues

```python
class SetValues(BaseOperation):
    """
    Replace all values in the input with a constant.

    Preserves the DataFrame structure (years, dimensions) but replaces
    all Value entries with the specified constant. Primarily useful
    with conditional execution (only_if/skip_if) to zero out data
    based on scenario parameters.
    """
    kind: Literal["set_values"] = "set_values"
    input: InputRef
    value: float
    unit: str | None = None  # If None, preserves input unit
```

Common patterns:

```yaml
# Zero out based on boolean parameter
- kind: set_values
  input: { port: emission_factor }
  value: 0
  only_if: { parameter: assume_zero_emissions }

# Set to a specific value (e.g., for testing or overrides)
- kind: set_values
  input: { port: baseline }
  value: 100
  unit: kt CO2/a
```

Unit behavior: If `unit` is specified, the output has that unit (must be compatible with input for dimensional analysis downstream). If `unit` is None, the output preserves the input's unit.

Absolute

```python
class Absolute(BaseOperation):
    """
    Take absolute value.
    """
    kind: Literal["abs"] = "abs"
    input: InputRef
```

Round

```python
class Round(BaseOperation):
    """
    Round values to specified decimal places.
    """
    kind: Literal["round"] = "round"
    input: InputRef
    decimals: int = 0
```

ScaleByReferenceYear

```python
```

```python
class ScaleByReferenceYear(BaseOperation):
    """
    Express all values relative to a reference year.

    Output[Y] = Input[Y] / Input[reference_year]
    Useful for index-style visualizations.
    """
    kind: Literal["scale_by_reference"] = "scale_by_reference"
    input: InputRef
    reference_year: int
```

Replaces `scale_by_reference_year`, `ratio_to_last_historical_value`.

---

6. Edge Transformations

Edge transformations handle structural changes at node boundaries. They are applied in order when data flows through an edge.

6.1 SelectCategoriesTransform

```python
class SelectCategoriesTransform(BaseModel):
    """
    Filter to specific categories within a dimension.
    """
    type: Literal["select_categories"] = "select_categories"
    dimension: Identifier
    categories: list[str]
    exclude: bool = False  # If true, exclude these categories instead
```

6.2 AssignCategoryTransform

```python
```

```python
class AssignCategoryTransform(BaseModel):
    """
    Add a dimension with a single fixed category.

    Useful when merging data from specialized nodes back into
    a unified dimensional structure.
    """
    type: Literal["assign_category"] = "assign_category"
    dimension: Identifier
    category: str
```

6.3 FlattenTransform

```python
class FlattenTransform(BaseModel):
    """
    Collapse a dimension by summing over all its categories.

    The dimension is removed from the output.
    """
    type: Literal["flatten"] = "flatten"
    dimension: Identifier
```

6.4 RenameDimensionTransform

```python
class RenameDimensionTransform(BaseModel):
    """
    Rename a dimension (for compatibility between nodes with
    different naming conventions).
    """
    type: Literal["rename_dimension"] = "rename_dimension"
    from_dimension: Identifier
    to_dimension: Identifier
```

Edge Transformation Union

```python
```

```
EdgeTransformation = Annotated[
    Union[
        SelectCategoriesTransform,
        AssignCategoryTransform,
        FlattenTransform,
        RenameDimensionTransform,
    ],
    Field(discriminator="type")
]
```

---

7. Formula Compilation

For convenience, users may write expressions that compile to operation pipelines.

7.1 Supported Syntax

```
expression := term (('+' | '-') term)*
term       := factor (('*' | '/') factor)*
factor     := base ('^' exponent)?
base       := NUMBER | IDENTIFIER | function_call | '(' expression ')'
function_call := IDENTIFIER '(' arguments ')'
arguments  := expression (',' expression)*
```

7.2 Supported Functions

Function	Compiles to
`cumsum(x)`	CumulativeSum
`cumprod(x)`	CumulativeProduct
`diff(x)`	Difference
`diff(x, n)`	Difference with periods=n
`shift(x, n)`	Shift
`clip(x, min, max)`	Clip
`abs(x)`	Absolute
`discount(x, rate)`	Discount

7.3 Compilation Example

Input formula:

```python
(port1 + port2) * cumsum(port3)
```

Compiled pipeline:

```yaml
operations:
  - kind: add
    inputs:
      - port: port1
      - port: port2
    output: _add_1

  - kind: cumsum
    input:
      port: port3
    output: _cumsum_1

  - kind: multiply
    inputs:
      - ref: _add_1
      - ref: _cumsum_1
    # No output = final result
```

7.4 Implementation Notes

- Use Python's `ast` module to parse (already in use)
- Generate deterministic intermediate names: `_optype_N`
- Track AST positions for error mapping
- Formulas are one-way compiled (not round-tripped from modified pipelines)

---

8. Pipeline Container and Validation

8.1 Pipeline Model

```python
```

```python
class OperationPipeline(BaseModel):
    """
    A complete pipeline definition for a node.
    """
    operations: list[Operation] = Field(min_length=1)

    @model_validator(mode='after')
    def validate_pipeline(self) -> Self:
        # Check that all intermediate references are valid
        defined_outputs: set[str] = set()
        for i, op in enumerate(self.operations):
            # Check inputs reference defined outputs or ports
            for input_ref in self._get_input_refs(op):
                if isinstance(input_ref, IntermediateRef):
                    if input_ref.ref not in defined_outputs:
                        raise ValueError(
                            f"Operation {i} references undefined intermediate '{input_ref.ref}'"
                        )

            # Track this operation's output
            if op.output:
                if op.output in defined_outputs:
                    raise ValueError(f"Duplicate output name '{op.output}'")
                defined_outputs.add(op.output)

            # Validate conditional execution modifiers
            if op.only_if is not None or op.skip_if is not None:
                if not self._is_single_input_operation(op):
                    raise ValueError(
                        f"Operation {i} ({op.kind}): only_if/skip_if can only be used "
                        f"with single-input operations"
                    )

        # Check exactly one operation has no output (the final one)
        final_ops = [op for op in self.operations if op.output is None]
        if len(final_ops) != 1:
            raise ValueError(
                f"Exactly one operation must have no output (the final result), "
                f"found {len(final_ops)}"
            )
        if final_ops[0] != self.operations[-1]:
            raise ValueError("The final operation (no output) must be last")

        return self

    def _get_input_refs(self, op: Operation) -> list[InputRef]:
```

```python
    """Extract all input references from an operation."""
    # Implementation depends on operation type
    ...

def _is_single_input_operation(self, op: Operation) -> bool:
    """
    Check if operation has exactly one primary input.

    Single-input operations: Clip, Coalesce, CumulativeSum, CumulativeProduct,
    Difference, Shift, RollingMean, TimeFilter, Interpolate, Extrapolate,
    FillMissingYears, SetValues, Absolute, Round, ScaleByReferenceYear, Negate

    Multi-input operations: Add, Subtract, Multiply, Divide, Power,
    SelectCategories (input + dimension), AggregateOver, SplitByShares,
    Broadcast, Where, Discount (input + rate), Annualize, CompoundGrowth
    """
    SINGLE_INPUT_KINDS = {
        'clip', 'coalesce', 'cumsum', 'cumprod', 'diff', 'shift',
        'rolling_mean', 'time_filter', 'interpolate', 'extrapolate',
        'fill_missing_years', 'set_values', 'abs', 'round',
        'scale_by_reference', 'negate'
    }
    return op.kind in SINGLE_INPUT_KINDS

def get_intermediate_ids(self) -> set[str]:
    """Return all intermediate output IDs."""
    return {op.output for op in self.operations if op.output}

def get_required_ports(self) -> set[str]:
    """Return all port IDs this pipeline requires."""
    ports = set()
    for op in self.operations:
        for ref in self._get_input_refs(op):
            if isinstance(ref, PortRef):
                ports.add(ref.port)
    return ports

def get_required_parameters(self) -> set[str]:
    """Return all parameter IDs this pipeline requires."""
    params = set()
    for op in self.operations:
        # Check conditional modifiers
        if op.only_if is not None:
            params.add(op.only_if.parameter)
        if op.skip_if is not None:
            params.add(op.skip_if.parameter)
        # Check input refs
```

```
        for ref in self._get_input_refs(op):
            if isinstance(ref, ParameterRef):
                params.add(ref.parameter)
    return params
```

8.2 Operation Union Type

```python
Operation = Annotated[
    Union[
        # Arithmetic
        Add, Subtract, Multiply, Divide, Power, Negate,
        # Clamping
        Clip, Coalesce,
        # Cumulative
        CumulativeSum, CumulativeProduct, Difference, Shift, RollingMean,
        # Time filtering
        TimeFilter, FillMissingYears,
        # Interpolation
        Interpolate, Extrapolate,
        # Dimensional
        SelectCategories, AggregateOver, SplitByShares, Broadcast,
        # Financial
        Discount, Annualize, CompoundGrowth,
        # Conditional
        Where,
        # Utility
        SetValues, Absolute, Round, ScaleByReferenceYear,
    ],
    Field(discriminator="kind")
]
```

---

9. Execution Model

9.1 Executor Interface

```python
```

```python
from abc import ABC, abstractmethod

class OperationExecutor(ABC):
    """Base class for operation executors."""

    @abstractmethod
    def execute(
        self,
        op: Operation,
        inputs: dict[str, PathsDataFrame],
        context: ExecutionContext,
    ) -> PathsDataFrame:
        """
        Execute the operation and return result.

        Args:
            op: The operation specification
            inputs: Map from input reference ID to resolved DataFrame
            context: Execution context (units, dimensions, metadata)

        Returns:
            The resulting PathsDataFrame with appropriate unit/dimensions

        Raises:
            UnitError: If units are incompatible
            DimensionError: If dimensions cannot be broadcast
            OperationError: For other execution failures
        """
        ...

class ExecutionContext(BaseModel):
    """
    Context available during pipeline execution.
    """
    model_year_range: tuple[int, int]  # (start_year, end_year)
    forecast_start_year: int
    available_dimensions: dict[str, list[str]]  # dimension -> categories

    # Parameter values from active scenario
    global_parameters: dict[str, Any]  # parameter_id -> value
    node_parameters: dict[str, Any]  # parameter_id -> value (node-local)

    def resolve_parameter(self, param_ref: ParameterRef) -> Any:
        """
        Resolve a parameter reference to its value.
```

```python
    Looks up node-local parameters first, then global parameters.
    Raises KeyError if parameter not found.
    """
    param_id = param_ref.parameter
    if param_id in self.node_parameters:
        return self.node_parameters[param_id]
    if param_id in self.global_parameters:
        return self.global_parameters[param_id]
    raise KeyError(f"Parameter '{param_id}' not found in scenario")
```

9.2 Pipeline Executor

```python
```

```python
class PipelineExecutor:
    """
    Executes a complete operation pipeline.
    """

    def __init__(self, executors: dict[str, OperationExecutor]):
        """
        Args:
            executors: Map from operation kind to executor instance
        """
        self.executors = executors

    def execute(
        self,
        pipeline: OperationPipeline,
        port_data: dict[str, PathsDataFrame],
        dataset_data: dict[str, PathsDataFrame],
        context: ExecutionContext,
    ) -> tuple[PathsDataFrame, dict[str, PathsDataFrame]]:
        """
        Execute the full pipeline.

        Args:
            pipeline: The pipeline to execute
            port_data: Input data from ports
            dataset_data: Referenced dataset data
            context: Execution context

        Returns:
            Tuple of (final_result, intermediates)
            where intermediates maps output IDs to their DataFrames
        """
        intermediates: dict[str, PathsDataFrame] = {}
        result: PathsDataFrame | None = None

        for op in pipeline.operations:
            # Check conditional execution
            should_skip, passthrough_input = self._check_conditional(op, context)

            if should_skip:
                # Skip operation, pass through primary input
                output = self._resolve_single_input(
                    passthrough_input, port_data, dataset_data, intermediates
                )
            else:
                # Resolve input references
```

```
            inputs = self._resolve_inputs(op, port_data, dataset_data, intermediates, context)

            # Execute operation
            executor = self.executors[op.kind]
            output = executor.execute(op, inputs, context)

        # Store result
        if op.output:
            intermediates[op.output] = output
        else:
            result = output

    assert result is not None
    return result, intermediates

def _check_conditional(
    self,
    op: Operation,
    context: ExecutionContext,
) -> tuple[bool, InputRef | None]:
    """
    Check if operation should be skipped due to only_if/skip_if.

    Returns:
        Tuple of (should_skip, passthrough_input)
        passthrough_input is the input to pass through if skipping
    """
    if op.only_if is not None:
        param_value = context.resolve_parameter(op.only_if)
        if not isinstance(param_value, bool):
            raise TypeError(f"only_if parameter must be boolean, got {type(param_value)}")
        if not param_value:
            return True, self._get_primary_input(op)

    if op.skip_if is not None:
        param_value = context.resolve_parameter(op.skip_if)
        if not isinstance(param_value, bool):
            raise TypeError(f"skip_if parameter must be boolean, got {type(param_value)}")
        if param_value:
            return True, self._get_primary_input(op)

    return False, None

def _get_primary_input(self, op: Operation) -> InputRef:
    """
    Get the primary input for passthrough during conditional skip.
```

```python
    Raises ValueError if operation has multiple inputs.
    """
    # Implementation inspects operation type and extracts single input
    # Raises error if operation has multiple inputs (e.g., Add, Multiply)
    ...

def _resolve_inputs(
    self,
    op: Operation,
    ports: dict[str, PathsDataFrame],
    datasets: dict[str, PathsDataFrame],
    intermediates: dict[str, PathsDataFrame],
    context: ExecutionContext,
) -> dict[str, PathsDataFrame | float]:
    """
    Resolve all input references to actual DataFrames or scalar values.

    For ParameterRef inputs (non-boolean, used as values), resolves to
    the parameter value which may be used as a scalar in calculations.
    """
    ...
```

9.3 Unit Validation

Unit compatibility is checked at execution time using pint:

```python
```

```python
def check_additive_compatibility(units: list[Unit]) -> Unit:
    """
    Check that all units can be added together.
    Returns the common unit.
    Raises UnitError if incompatible.
    """
    if not units:
        raise ValueError("No units to check")

    base = units[0]
    for u in units[1:]:
        # Check pint dimensional compatibility
        if not pint_compatible(base.unit, u.unit):
            raise UnitError(f"Cannot add {base.unit} and {u.unit}")

        # Check specifier compatibility (if present)
        if base.specifier and u.specifier and base.specifier != u.specifier:
            raise UnitError(
                f"Cannot add {base.unit} ({base.specifier}) "
                f"and {u.unit} ({u.specifier})"
            )

    return base
```

---

10. Integration with Existing System

10.1 Node Definition

Nodes with operation pipelines use a new field:

```python
```

```python
class NodeSer(IdentifiedModel):
    # ... existing fields ...

    # Either type OR operations must be specified, not both
    type: str | None = None
    "Python class for typed nodes"

    operations: OperationPipeline | None = None
    "Pipeline operations for computed nodes"

    @model_validator(mode='after')
    def check_type_or_operations(self) -> Self:
        if self.type and self.operations:
            raise ValueError("Node cannot have both type and operations")
        if not self.type and not self.operations:
            raise ValueError("Node must have either type or operations")
        return self
```

10.2 Database Storage

Using django-pydantic-field or similar:

```python
from django_pydantic_field import SchemaField

class Node(models.Model):
    id = models.CharField(max_length=100, primary_key=True)
    name = SchemaField(schema=I18nString)
    operations = SchemaField(schema=OperationPipeline, null=True)
    # ... other fields ...
```

10.3 Migration from FormulaNode

Existing FormulaNode instances can be migrated:

1. Parse existing formula string
2. Compile to OperationPipeline
3. Store pipeline, optionally preserving formula as comment
4. Deprecate FormulaNode class over time

---

11. UI Integration

11.1 Graph Editor

The planned graph-based editor should:

- Display pipeline operations as a mini-graph within a node
- Show intermediate values on hover during debugging
- Allow drag-and-drop addition of operations
- Validate in real-time as user edits

11.2 Intermediate Value Display

For explainability, the UI can show:

```
┌──────────────
```

```python
@mcp_tool
def inspect_pipeline_execution(
    node_id: str,
    year: int | None = None,
) -> PipelineInspection:
    """
    Return detailed execution trace for a node's pipeline.

    Shows each operation with its inputs, outputs, and intermediate values.
    """
    ...
```

---

12. LLM Integration

12.1 Schema for Prompts

Export JSON Schema for LLM prompting:

```python
schema = OperationPipeline.model_json_schema()
```

12.2 Example Prompt Structure

```
You are helping build a climate impact model. Define an operation pipeline
that calculates net emissions from these inputs:

Input ports:
- gross_emissions (unit: kt CO2/a, dimensions: [sector])
- reduction_transport (unit: kt CO2/a, dimensions: [])
- reduction_buildings (unit: kt CO2/a, dimensions: [])

Requirements:
- Subtract both reductions from gross emissions
- Ensure result is non-negative
- Output should have sector dimension

Respond with a valid YAML operations list following this schema:
{schema}
```

12.3 Validation Feedback Loop

When LLM-generated pipelines fail validation:

1. Capture Pydantic validation error
2. Format as natural language feedback
3. Request corrected pipeline
4. Iterate until valid

---

13. Open Questions

13.1 Error Margins and Uncertainty

The current spec supports a `Forecast` boolean. Future extensions might include:

- `error_margin: float | None` - Uncertainty bounds
- `data_quality: Literal["measured", "estimated", "projected"]`
- `source: str | None` - Data provenance

Should uncertainty propagate through operations? (Adds complexity.)

13.2 Iteration / Convergence

Some models might need iterative solving (e.g., equilibrium models). This is out of scope for the initial implementation but might be needed for Phase 2.

13.3 Sub-annual Resolution

Currently Year is integer (annual). Future might need:

- Quarterly: `2024Q1`
- Monthly: `2024-01`
- Custom periods

This affects cumulative operations and would need careful design.

13.4 Lazy vs Eager Execution

Current design assumes eager execution (all intermediates computed). For very large models, lazy execution with result caching might be beneficial.

---

14. Implementation Roadmap

Phase 1: Core Types and Execution

1. Define all Pydantic models
2. Implement PipelineExecutor
3. Implement executors for arithmetic operations
4. Add unit tests with property-based testing

Phase 2: Migration Infrastructure

1. Build FormulaNode → Pipeline compiler
2. Create migration scripts for existing models
3. Add database schema support
4. Deprecation warnings for FormulaNode

Phase 3: Advanced Operations

1. Financial operations (discount, annualize)
2. Conditional operations (where)
3. Complex interpolation/extrapolation

Phase 4: UI Integration

1. Pipeline visualization in graph editor
2. Intermediate value inspection
3. Drag-and-drop pipeline building
4. Real-time validation

Phase 5: LLM Integration

1. Schema export for prompts
2. Validation feedback formatting
3. Example generation from existing pipelines
4. Testing with Claude/GPT for pipeline authoring

Appendix A: Migration from Current Operations

Operations to Keep (rename for clarity)

Current	New
add	add
multiply	multiply
cumulative	cumsum
cumulative_product	cumprod
difference	diff
forecast_only	time_filter (mode=forecast_only)
inventory_only	time_filter (mode=historical_only)
extrapolate	extrapolate

Operations to Compose from Primitives

Current	Replacement
make_nonnegative	clip(min=0)
make_nonpositive	clip(max=0)
empty_to_zero	coalesce(0)
arithmetic_inverse	negate
geometric_inverse	divide(1, x)
complement	subtract(1, x)
complement_cumulative_product	subtract(1, cumprod(x))
absolute	abs

Operations to Remove (workarounds)

Current	Action
skip_dim_test	Fix underlying dimension handling
ignore_content	Remove; find proper solution
other	Remove; use explicit operations
do_correction	Remove; model corrections explicitly

Appendix B: Complete YAML Example

```yaml
```

```yaml
# Example: Transport emissions calculation with scenario parameters

# Global parameters (defined at model level)
parameters:
  - id: discount_rate
    type: number
    default: 0.03
    min: 0
    max: 0.1
    label_en: "Discount rate"

  - id: green_electricity_emissionless
    type: bool
    default: false
    label_en: "Treat green electricity as zero-emission"

nodes:
  - id: vehicle_kilometers
    name_en: Vehicle kilometers traveled
    input_ports:
      - id: per_capita_vkt
        quantity: vehicle_mileage_per_capita
        unit: km/person/a
      - id: population
        quantity: population
        unit: person
      - id: mode_shares
        quantity: share
        unit: dimensionless
        required_dimensions: [transport_mode]
    output_ports:
      - id: output
        quantity: vehicle_mileage
        unit: Mkm/a
        dimensions: [transport_mode]
    operations:
      - kind: multiply
        inputs:
          - port: per_capita_vkt
          - port: population
        output: total_vkt
        description: "Total VKT from per-capita and population"

      - kind: multiply
        inputs:
          - ref: total_vkt
```

```yaml
      - port: mode_shares
    output: vkt_by_mode
    description: "Distribute across transport modes"

  - kind: divide
    numerator:
      ref: vkt_by_mode
    denominator:
      value: 1000000
      dimensionless: true
    description: "Convert km to Mkm"

- id: grid_emission_factor
  name_en: Grid electricity emission factor
  input_ports:
    - id: base_ef
      quantity: emission_factor
      unit: g CO2/kWh
  output_ports:
    - id: output
      quantity: emission_factor
      unit: g CO2/kWh
  operations:
    # If green electricity assumption is enabled, zero out the EF
    - kind: set_values
      input: { port: base_ef }
      value: 0
      only_if: { parameter: green_electricity_emissionless }
      description: "Zero emissions if green electricity assumption active"

- id: transport_emissions
  name_en: Transport CO2 emissions
  input_ports:
    - id: vkt
      quantity: vehicle_mileage
      unit: Mkm/a
      required_dimensions: [transport_mode]
    - id: emission_factors
      quantity: emission_factor
      unit: g CO2/km
      required_dimensions: [transport_mode]
  output_ports:
    - id: output
      quantity: emissions
      unit: kt CO2/a
      dimensions: [transport_mode]
  operations:
```

```yaml
      - kind: multiply
        inputs:
          - port: vkt
          - port: emission_factors
        output: emissions_grams
        description: "VKT × emission factor"

      - kind: divide
        numerator:
          ref: emissions_grams
        denominator:
          value: 1000000000
          unit: g/kt
        description: "Convert g to kt"

  - id: transport_costs_npv
    name_en: Transport investment costs (NPV)
    input_ports:
      - id: annual_costs
        quantity: cost
        unit: EUR/a
    output_ports:
      - id: output
        quantity: cost
        unit: EUR
    operations:
      # Use discount rate from scenario parameter
      - kind: discount
        input: { port: annual_costs }
        rate: { parameter: discount_rate }
        base_year: 2024
        output: discounted
        description: "Discount future costs to present value"

      - kind: cumsum
        input: { ref: discounted }
        description: "Sum to get NPV"

edges:
  - id: vkt_to_emissions
    source:
      node: vehicle_kilometers
      port: output
    sink:
      node: transport_emissions
```

```
  port: vkt
transformations: []
```

End of specification